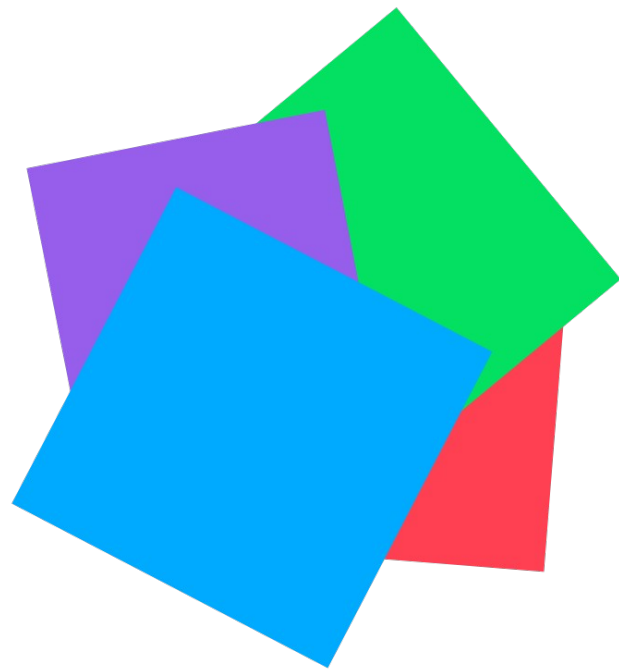


Практика 4: две фичи с агентом

Смотрим показ, повторяем в своём проекте,
проверяем результат каждого этапа.



Наш маршрут на 90 минут

1 Выбираем две маленькие фичи · 5 мин

Ожидаемое поведение и способ проверки

2 Готовим среду для А · 20 мин

AGENTS.md, один skill, один MCP

3 Реализуем и принимаем А · 15 мин

Короткое поручение, diff, тест, commit

4 Добавляем правила и hooks · 15 мин

Style guide и автоматическая проверка

5 Делаем В отдельно · 25 мин

Worktree, новый контекст и subagent review

6 Объединяем и передаём · 10 мин

Общая проверка, handoff, итоги и ДЗ

Этап 1 · Выбираем две маленькие фичи

Откройте свой проект из практик 1–3. Ведущий показывает AI-review API, вы выбираете сопоставимый кусок своей системы.

A: входная валидация

Проверить входной diff.
Большой запрос отклонить.
При отказе не вызывать LLM.

B: ошибка зависимости

Ограничить ожидание LLM.
Вернуть понятную ошибку.
Сохранить сценарии A.

[Пошаговый маршрут и учебный каркас](#)

Учебный AI-review API показывает масштаб задачи. Работайте со своим проектом и согласованным контрактом.

Этап 2

Готовим среду для А

20 минут · Можно дать короткое поручение.

Сейчас: AGENTS.md, подходящий skill и MCP.

Готово, когда: Правило прочитано, skill загружен, tool вызван.

2.1 · Правила проекта в AGENTS.md

В корне проекта укажите, где требования, как проверять результат и что нельзя менять.

AGENTS.md · минимальное содержание

Контракт: docs/requirements.md
Пример теста: tests/test_example.py
Проверка: sh scripts/check.sh
Не меняй контракт и runner без поручения.

В чате OpenCode

Прочитай AGENTS.md. Найди контракт и тест-пример. Назови ограничения фичи А.

Проверка: Агент прочитал нужные файлы и знает реальную команду проверки.

[OpenCode: правила](#)

Замените примерные пути на существующие файлы своего проекта.

2.2 · Где искать полезные skills

Выберите один пакет под фичу А. В нашем демо — TDD для кода и тестов.

skills.sh · каталог

Поиск по задаче, библиотеке или инструменту.

<https://skills.sh/>

Anthropic Skills · примеры пакетов

Смотрите инструкции, скрипты и зависимости.

<https://github.com/anthropics/skills>

test-driven-development

Для фичи: тест, ожидаемое падение, реализация.

<https://github.com/obra/superpowers/tree/main/skills/test-driven-development>

webapp-testing

Для работающего UI: сценарий и проверка браузером.

<https://github.com/anthropics/skills/tree/main/skills/webapp-testing>

Проверка: Можете объяснить, какой шаг вашей задачи улучшит выбранный skill.

2.3 · Куда положить skill в OpenCode

Скопируйте папку skill в проект вместе со связанными ресурсами.

Папка в проекте

```
.opencode/skills/test-driven-development/  
SKILL.md  
writing-good-tests.md
```

Во втором терминале проекта

```
opencode api skill.list \  
-param "location[directory]=$PWD"
```

В чате OpenCode

Загрузи test-driven-development и прочитай writing-good-tests.md.
Примени процедуру к A.

Проверка: Виден вызов skill и чтение нужного reference, а не только папка на диске.

[OpenCode 2: skills](#)

[SKILL.md · проверенный снимок](#)

[writing-good-tests.md](#)

В 2.0.20 команды debug skill нет. При пустом первом списке дождитесь загрузки проекта и повторите запрос.

2.4 · Где искать MCP под свою задачу

Начните с одного полезного подключения. Для документации библиотек подойдёт Context7.

MCP Registry · официальный реестр

Найти сервер и перейти к инструкции автора.

<https://registry.modelcontextprotocol.io/>

Context7 · документация

Примеры нужного SDK или framework; сверяем версию.

<https://github.com/upstash/context7>

Playwright MCP · браузер

Проверить пользовательский сценарий в работающем UI.

<https://github.com/microsoft/playwright-mcp>

GitHub MCP · задачи и PR

Прочитать issue, diff или обсуждение с нужными правами.

<https://github.com/github/github-mcp-server>

Проверка: Есть конкретный вопрос или действие, ради которого подключаем сервер.

2.5 · Подключаем Context7

Добавьте MCP-настройку в `opencode.json` в корне проекта.

`opencode.json` · OpenCode 2

```
{
  "mcp": { "servers": {
    "context7": {
      "type": "remote",
      "url": "https://mcp.context7.com/mcp",
      "disabled": false,
      "codemode": false
    }
  }}
}
```

В терминале

`opencode mcp list`

В чате OpenCode

Найди пример валидации для версии FastAPI в проекте. use context7

Проверка: После `connected` видим настоящий `query-docs` и полезный ответ.

[OpenCode 2: MCP](#)

[Context7](#)

Добавьте секцию в существующий JSON. После изменения загрузите конфигурацию заново: `opencode reload`.

Этап 3 · Поручаем агенту фичу А

Переключаемся на экран OpenCode. Даём результат, границы и проверку.

Сообщение в OpenCode

Реализуй фичу А по требованиям проекта.
Используй выбранный skill и найденные примеры.
Сначала покажи падающую проверку, затем исправление.
Покажи diff и фактически выполненные команды.
Не делай commit без моей приёмки.

Проверка: Проверяемый сценарий А проходит. Видно, какие действия выполнил агент.

3.1 · Принимаем и сохраняем А

Сначала читаем diff и результат проверки. Затем сохраняем принятую версию.

В терминале · пример для нашего API

```
sh scripts/check.sh
git diff -check
git diff
git status -short
git add app tests
git commit -m "feat: implement feature A"
```

Проверка: А проверена и сохранена вместе с настройками до worktree В.

app, tests и check.sh — пути демо. В своём проекте выберите нужные файлы и действующие проверки.

Этап 4

Усиливаем среду

15 минут · Правило можно проверить кодом.

Сейчас: Style guide и автоматический запуск проверок.
Готово, когда: Hook показал сбой и успешный повтор.

4.1 · Добавляем короткий style guide

Сохраните 3–5 правил, которые нужны именно вашей фиче, и один хороший пример.

[docs/style-guide.md](#) · пример

Проверяй поведение через публичный интерфейс.
Используй типы и соглашения проекта.
Не добавляй framework ради маленькой правки.
Не ослабляй runner ради зелёного результата.

Связать с AGENTS.md

Перед изменением кода прочитай [docs/style-guide.md](#).

Показать применение

Попросите агента назвать правило и место в коде, где оно выполнено.

Проверка: Правило связано с реальным примером и действующей проверкой.

[OpenCode 2: инструкции и миграция](#)

В 2.0.20 instructions не загружает файл. Ссылка в AGENTS.md и явное чтение устраняют эту неоднозначность.

4.2 · Hook проверяет работу после правки

Плагин OpenCode 2 регистрирует `ctx.tool.hook("execute.after", ...)`. Он запускает один доверенный runner.

Сохраните пример плагина по этому пути

```
.opencode/plugins/check-after-edit.js
```

Сначала вручную

```
sh scripts/check.sh
```

Команда должна работать до подключения hook.

Затем в OpenCode

```
opencode reload
```

Измените временный тест:
событие даёт FAIL, исправление даёт PASS.

Проверка: Runner запустился по событию, а его результат вернулся агенту.

[Готовый hook для OpenCode 2](#)

[Миграция plugins V1 → V2](#)

V1 `tool.execute.after`-плагины в OpenCode 2 не исполняются. Используйте обновлённый пример.

Бонус: свой субагент под проект

Между фичами А и В, если успеваете. Выберите тестировщика, reviewer или исследователя.

Что настраиваем	Пример для тестировщика
Задача	Проверить фичу В по требованиям.
Контекст	Описание В, diff, тесты, команда запуска и адрес сервиса.
Процедура	Выбрать сценарии, выполнить проверки, сравнить ожидание и факт.
Инструменты и права	Читать проект, запускать нужные проверки. Самостоятельно продукт не исправлять.
Результат	Проблемы, доказательства и непроверенные сценарии.

Можно подключить testing- или review-skill под выбранную роль.

Необязательный этап. На оценку не влияет; можно пропустить и перейти к фиче В.

Бонус · Создаём своего tester

Сейчас сохраняем профиль в проекте. Вызываем его после реализации В.

В терминале проекта

```
mkdir -p .opencode/agents  
# Сохраните tester.md из шаблона ниже  
opencode reload  
opencode debug agents
```

Файл в проекте

.opencode/agents/tester.md
mode: subagent, инструкции и права.

Перед worktree В

Убедитесь, что tester обнаружен.
Сохраните профиль и настройки в commit.

Проверка: В списке есть tester (subagent), а профиль сохранён вместе с проектом.

[Шаблон tester](#)

[OpenCode 2: агенты](#)

В OpenCode 2.0.20 мастера agent create нет. Создаём профиль файлом; вызываем после реализации В.

Этап 5

Делаем В отдельно

25 минут · Вторая фича готова к объединению.

Сейчас: Worktree, короткое поручение и subagent review.
Готово, когда: В проверена; А сохранила поведение.

5.1 · Создаём worktree для В

Сохраните А и настройки в commit: AGENTS, skill, MCP, style guide, hook, runner и профиль, если он есть.

После commit · имена ветки и каталога должны быть свободны

```
git worktree add -b practice-04-b ../project-p4-b HEAD
cd ../project-p4-b
pwd
git status -short
# Подготовьте зависимости этой рабочей копии
opencode
```

Проверка: Агент открыт в каталоге В. Правила, skill, MCP и hook доступны.

[Git worktree](#)

Worktree не переносит .venv и не изолирует порты, БД или права процесса.

5.2 · Делаем В и получаем отдельное review

Новый чат получает актуальные правила. После изменения передаём проверку другому исполнителю.

Сообщение 1 · реализация В

Реализуй В по требованиям проекта.
Прочитай AGENTS.md и style guide.
Сохрани сценарии А.
Покажи результаты проверок.

После завершения В

@explore Проверь diff В по требованиям. Назови подтверждённые проблемы. Файлы не меняй.

Проверка: Виден отдельный вызов subagent и ваше решение по его замечаниям.

[OpenCode 2: агенты](#)

Если создали tester, используйте @tester с поручением запустить проверки.

Этап 6

Объединяем и передаём

10 минут · Понятно, что сделано и что дальше.

Сейчас: Общая проверка, handoff и объяснение ДЗ.

Готово, когда: Есть итоговый diff, проверки и следующее поручение.

6.1 · Объединяем и снова проверяем

В В: примите diff и сделайте commit. Затем вернитесь в исходный каталог проекта.

В терминале исходного проекта

```
git merge -ff-only practice-04-b  
sh scripts/check.sh  
git status -short
```

Проверка: Проверено объединённое состояние, а не только две отдельные ветки.

6.2 · Передаём контекст новой сессии

Сначала сохраните короткий docs/HANDOFF.md: состояние, источники, проверки и ограничения.

Сообщение в новом чате OpenCode

Прочитай AGENTS.md и docs/HANDOFF.md.
Что уже сделано и что осталось?
Запусти проверку проекта.
Пока ничего не меняй.

Проверка: Новая сессия корректно восстановила состояние и выполнила проверку.

Домашнее продолжение и сдача

Для самостоятельной работы. Можно продолжить проект или выбрать другую задачу; сделанное на занятии засчитывается.

1 Skill: создание или исследование

Содержательная автоматизация либо запуск готового skill и собственный короткий разбор.

2 Собственный MCP

Один полезный tool, реальный вызов, успешный сценарий и обработка ошибки.

3 Свободная рефлексия

Что помогло, что мешало и где пришлось вмешаться. Без заданного объёма и «правильного» мнения.

Домашнее задание: 10 баллов

Оцениваются реальные действия и объяснение. Количество подключений само по себе баллов не добавляет.

А. Настроенная среда

5 баллов

Покажите применение правил, skill, MCP и автоматической проверки.

По 1 б: AGENTS.md, skill, MCP, hook, обоснование подключений.

Б. Skill: создание или исследование

2 балла

1 б: устройство или автоматизация. 1 б: запуск и проверенный результат.

Разбор готового skill: 3–4 предложения самостоятельно, своими словами.

В. Собственный MCP

2 балла

1 б: полезный tool. 1 б: вызов и результат, включая ошибочный вход.

Г. Свободная рефлексия

1 балл

reflection.md с конкретными наблюдениями. Свободный формат.

Сдача: HTML предпочтителен; Markdown по блокам тоже допустим. Приложите файлы и подтверждения.

Поиск по коду: что доступно в нашей версии

Ограничение LSP в OpenCode 2

Используем штатные lint, typecheck и тесты через check.sh. Агент получает реальные ошибки.

AST-index: исходники

Можно подключить для поиска символов в большом проекте. Нужны установка, индекс и инструкция агенту.

OpenCode 2.0.20: LSP-конфигурация принимается, но language servers, LSP tools и diagnostics пока не работают.

Что вы теперь можете показать

Две небольшие итерации и среда, которую можно использовать дальше.

Правила и skill применены. MCP дал полезный ответ. Hook проверил изменение. В сделана отдельно и принята вместе с A.

Назовите один механизм, который помог, и одно место, где потребовалось ваше решение.

Что сделали сами? Что продолжите дома?